Gustave Eiffel University

Internship Report
Master 2 Bézout

Fine-Grained Analysis of Hyperscan Engine

CHAU Dang Minh

Supervisors: Prof. Cyril NICAUD (LIGM)
Dr. Pablo ROTONDO (LIGM)

Paris, July 2026

Contents

List of Figures **3**

1 Introduction **4**

2 An Overview of the Hyperscan Engine **5**

2.1 Compilation 5

2.2 Execution 7

2.3 The FDR Matcher 7

2.4 Related Problems 8

2.4.1 Optimizing regular expression decomposition 8

2.4.2 Analysis of the FDR string matcher 8

2.4.3 Statistics of regular expressions 8

3 The Length of Longest Literals **9**

A Appendix **10**

A.1 The Shift-Or algorithm 10

A.2 BST-like distributions of regular expression trees 10

B Bibliography **11**

List of Figures

2.1 Dependency graph of main functions in the compilation phase. 6

2.2 The dependency graphs by Violet and Rose. 6

2.3 Dependency graph of main functions in the execution phase. 7

2.4 Proposed sequential dependency graph representation for `abc[0-9]+xyz`. 8

1

Introduction

Since Knuth's anouncement of the term *analysis of algorithms* [3, Preface], the field has grown to include highly specialized subfields. A basic classification is based on two aspects. The first aspect is the scenario under which the performance of an algorithm is analyzed, which concludes worst-case and average-case analysis. In general, average-case analysis is more informative than worst-case analysis, and is often more challenging to perform, since it requires an probabilistic model of the input that is tractable, yet expressive enough. For example, in the case of regular languages, it may not be a good idea to generate their syntax trees uniformly at random [4]. Interestingly, average-case analysis opens up the door to apply techniques from other fields, such as probability theory and analytic combinatorics, and raises new question, such as those about the limiting objects when the input size tends to infinity. The second aspect is the underlying underlying cost model, which can be the uniform or word-RAM model. The main purpose is to align quantitative analysis with practical performance, demonstrated by execution time. Moderns analysis of modern algorithms requires new cost models to natively capture hardware features, such as caching, branch prediction and parallelism.

In this intership, we study Hyperscan a regular expression matching engine that leverages SIMD parallelism to achieve high performance. Acknowledging the sophisticated design of Hyperscan, we proposed related questions and attempted to answer them. Hyperscan works by extracting possible strings from a regular expression, and then matching them once in parallel by its SIMD string matchers. Hence it is natural to analyze statistics related to those strings under a distribution. In particular, our question is to understand the asymptotic of the length of the longest strings extracted from a regular expression. Other questions are also posed clearly although not studied to a considerable extend. Details are given in the next chapter.

The rest of the report is organized as follows. In **Chapter 2**, we present Hyperscan and problems arised in the analysis. **Chapter 3** contains our results on the question of the length of the longest strings. Before each proof, we present the motivation and reasoning behind our approaches, which the reader can optionally skip and then go back after reading the proof. We place details of popular theoretical and experimental techniques in **Chapter 4**, where the reader can refer to for background knowledge. Finally, we conclude in **Chapter 5** with a summary of our work and future directions.

2

An Overview of the Hyperscan Engine

The regular expression matching engine Hyperscan [7] was primarily developed by the startup company Sensory Networks in 2008, and was later acquired and open-sourced by Intel in 2013. Closed source in 2023 at version 5.4, it is forked into Vectorscan [6] by VectorCamp to support more architectures like ARM. Since Vectorscan is still being actively maintained publicly, analyzing its core Hyperscan functionality still provides valuable insights into the design and optimization of high-performance regular expression matching engines, interesting to both academia and industry.

Hyperscan is designed to match a set of regular expressions against a stream of data, with its primary use case being deep packet inspection. The engine leverages SIMD (Single Instruction, Multiple Data) features supported by modern CPUs, which uses longer registers to process multiple similar operations in one instruction. Architecturally, it operates in two distinct phases: the compilation phase and the execution phase. It supports two modes of operation: block mode and stream mode. In block mode, the engine processes a single block of data at a time, while in stream mode, it maintains state across multiple blocks, allowing for continuous matching over a data stream. We focus on the block mode.

2.1 Compilation

An overview of the compilation process is illustrated in Figure 2.1. The compilation phase provides central APIs, such as the `hs_compile` function. The function accepts a set of regular expressions. A facade object of class `NG` is initialized, which consumes a compile context of type `CompileContext`. Through the function `addExpression`, each expression is converted into a Glushkov NFA [1] (through `buildGraph`) and added to the facade (through `NG.addGraph`).

In `NG.addGraph`, the NFA is firstly decomposed base on its language through `calcComponents`. If its language is L, it may be split into NFAs whose languages are $L_1, L_2, \ldots, L_n$ such that

$$L = L_1 \cup L_2 \cup \ldots \cup L_n.$$

The components are added one by one through `addComponent`, which then passed to the Violet subengine through the `doViolet` function, which in turn calls `doInitialVioletTransform` and `RoseBuild.addRose`.

In `doInitialVioletTransform`, the graph analyses described in the Hyperscan paper—namely dominant path, dominant region, and network flow analysis—are performed. These analyses decompose the NFA

into literals and sub-FAs, yielding a dependency graph that dictates the matching order. At this phase (`RoseInGraph`), each literal is a vertex and each sub-FA acts as an edge. This graph is then passed to the Rose subengine for deduplication of FAs, compilation into bytecode, and execution orchestration. Notably, in the subsequent (`RoseGraph`) representation, the sub-FAs are instead attached directly to their associated literal vertices via `left` or `suffix` properties. Multiple FAs may be merged into an FA with multiple initial states. Therefore, the `suffix` property also contains a sub-property `top` indicating which initial state should be chosen.

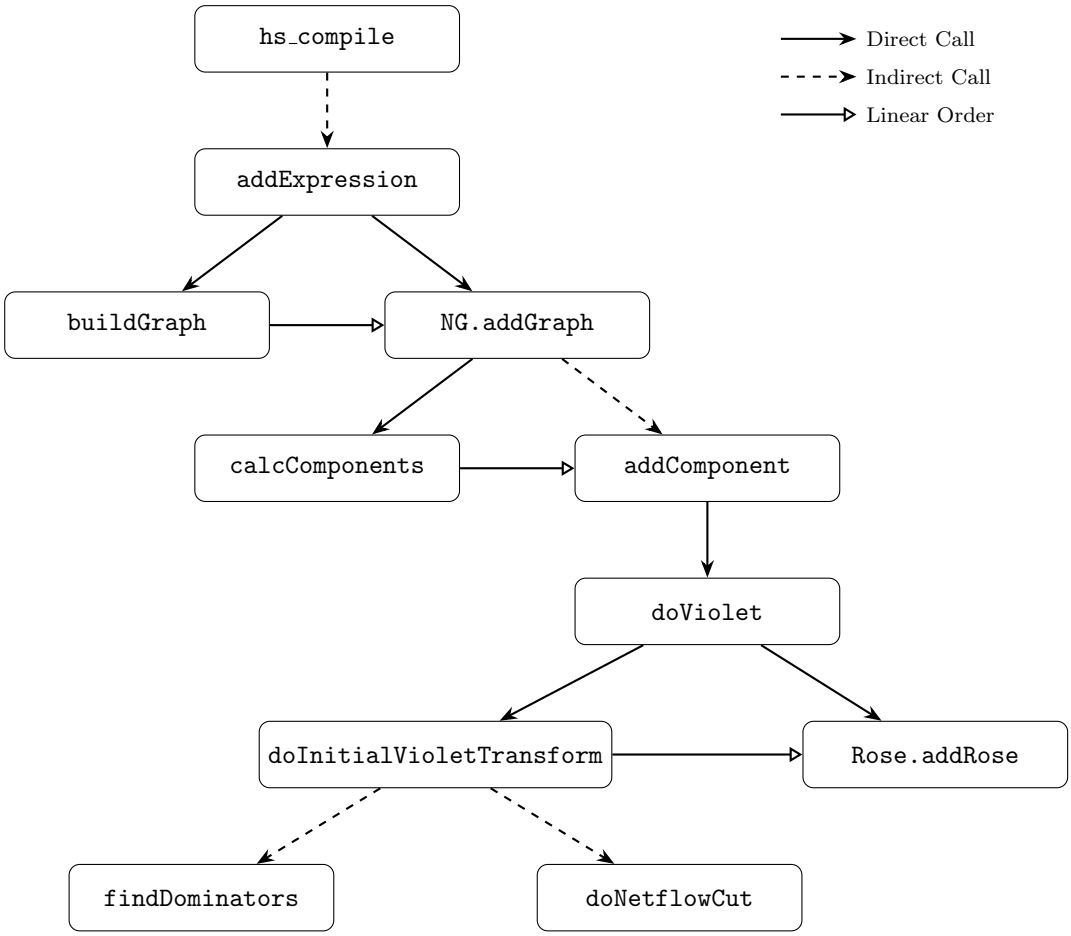
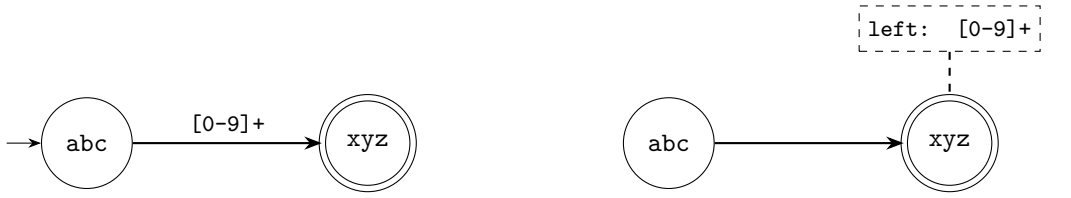


Figure 2.1: Dependency graph of main functions in the compilation phase.



(a) `RoseInGraph` (Sub-FA as Edge)

(b) `RoseGraph` (Sub-FA as Vertex Property)

Figure 2.2: The dependency graphs by Violet and Rose.

Consider an example of a regular expression `abc[0-9]+xyz`. If we discard heuristics for short literals, then the graphs returned by Violet and Rose are as in Figures 2.2a and 2.2b, respectively. They both mean that if there is a match of `abc` that *ends* at position i (excluded) and a match of `xyz` that *starts* at position j (included), such that $j - i > 0$, then the subtext at $[i, j)$ must match the sub-FA `[0-9]+`. Further optimization imposes that

$$m \leq j - i \leq M,$$

where m and M are the minimum and maximum *widths* of the sub-FA, i.e., the minimum and maximum lengths of possible strings that match the sub-FA.

2.2 Execution

An overview of the execution phase is illustrated in Figure 2.3. The process begins at `hs_scan`. There are three execution scenarios already determined by Rose: (1) the set of patterns are all literals (2) there are no literals that wake up FAs, and (3) literals wake up FAs. The last scenario is the most common. Its flow reaches roseBlockExec, where now one of its string matchers is called through hwlmExec together with a roseCallback function. The `roseCallback` function eventually calls roseRunProgram. Here is where the `left` or `suffix` FAs are checked, waken up and executed.

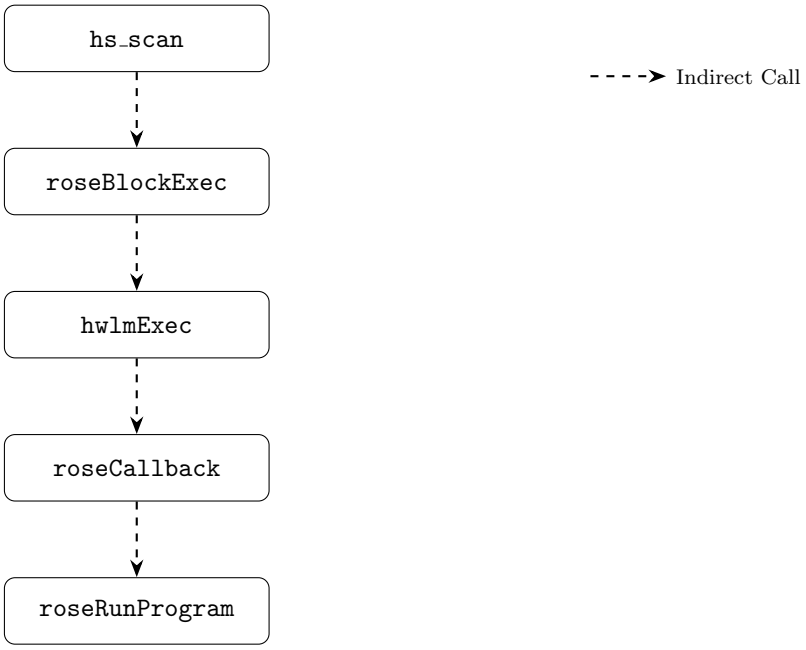


Figure 2.3: Dependency graph of main functions in the execution phase.

2.3 The FDR Matcher

There are at least four SIMD-based string matchers in Hyperscan: Noodle is used when there is a single literal, Teddy [5] or Harry [8] when there are 2 to 300, and FDR when there are more than 300. The last three matchers are all prefilter-based. Firstly, they group the literals into a set of *buckets*. Each bucket can be regarded as an indeterminate string (see e.g., [2])

Definition 2.1. Let Σ be an alphabet. An indeterminate string P of length n over Σ is a sequence of n non-empty sets of characters from Σ, i.e.,

$$\boldsymbol{P} = \boldsymbol{p}_1\boldsymbol{p}_2\dots\boldsymbol{p}_n,$$

where $\emptyset \neq p_i \subseteq \Sigma$ for all $i \in [1, n]$. The subset Σ is usually referred to as the *don't care* character. A string T over Σ matches the indeterminate string $\boldsymbol{P}$ if and only if $|T| = n$ and $T[i] \in \boldsymbol{p}_i$ for all $i \in [1, n]$. ■

Let us remark that the set of indeterminate strings over Σ is a strict subset of the set of star-free regular expressions over Σ. For example, the regular expression `ab|cd` is not an indeterminate string. Let

$\{P_1, P_2, \ldots, P_k\}$ be the set of literals in a bucket. Let p_{ij} be the j-th character of literal P_i. The literals are right-aligned and padded with the don't care character $*$ to form an indeterminate string $\boldsymbol{P}$ of length $n = \max\{|P_1|, |P_2|, \ldots, |P_k|\}$, i.e., $\boldsymbol{P} = \boldsymbol{p}_1\boldsymbol{p}_2 \ldots \boldsymbol{p}_n$, where

$$\boldsymbol{p}_{n-j} = \begin{cases} \{p_{i,|P_i|-j} \mid 1 \le i \le k, j \le |P_i|\}, & \text{if } p_{i,|P_i|-j} \text{ is defined for every } i, \\ \Sigma, & \text{otherwise.} \end{cases}$$

Let T be a text. The string matchers attempt to check if T matches $\boldsymbol{P}$, and if so, which literal P_i is matched. The FDR matcher is an SIMD extension of the Shift-Or algorithm (Appendix A.1).

2.4 Related Problems

2.4.1 Optimizing regular expression decomposition

Both representations in Figure 2.2 are likely suboptimal for theoretical analysis. Firstly, the sub-FA may appear several times, even though it only appear once in the regular expression, for example in the case of `(abc|def)[0-9]+xyz`. Secondly, although we had to mark `xyz` as a terminal vertex in Figure 2.2, this is not strictly accurate: the matching process terminates at the end of the sub-FA, not at the end of `xyz`. If there are remaining FAs or literals to the right, it is the sub-FA for `[0-9]+` that will determine the next matching position. Therefore, we propose a more convenient representation as follows. Each vertex is either a literal or a sub-FA. There are three types of directed edges.

- Edges that uses the *end* positions of the matches of the source vertex to determine the *start* positions of the matches of the target vertex. They connect two literals or a literal and a sub-FA. We call this type of edge `setStart`.
- Edges that uses the *start* positions of the matches of the source vertex to determine the *end* positions of the matches of the target vertex. They connect a literal and a sub-FA. We call this type of edge `setEnd`.
- Edges that acts as a pure trigger, without any position information. They connect two sub-FAs. We call this type of edge `trigger`.

Consider another example of `abc[0-9]+xyz(de)*`. In its dependency graph in the new representation is as in Figure 2.4.

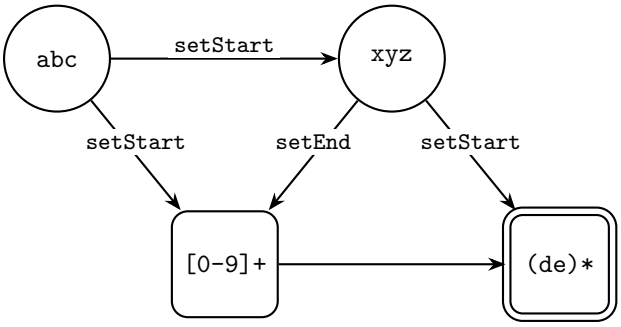


Figure 2.4: Proposed sequential dependency graph representation for `abc[0-9]+xyz`.

2.4.2 Analysis of the FDR string matcher

2.4.3 Statistics of regular expressions

3

The Length of Longest Literals

In this chapter, we examine the length of the longest literals

A

Appendix

A.1 The Shift-Or algorithm

Let Σ be the alphabet, T be the text, and P be the pattern. Suppose that we have registers of length w bits. This requires that $|P| \leq w$. As usual, we index characters in the pattern P and T from left to right, starting from 0. But for the registers, it is convenient to index bits from right to left (LSB), starting from 0.

In the compile stage, we construct masks for each character in the alphabet, which we denote by a function $M : \Sigma \to \{0,1\}^w$. For each character $c \in \Sigma$, we have

$$\begin{aligned} P[i] = c \iff M[c][i] = 0, \quad & i \in [0, |P| - 1] \\ M[c][i] = 1, \quad & i \in [|P|, w - 1] \end{aligned} \tag{1.1}$$

The execution stage is as follows. We initialize a state register S to all 1s ($S \leftarrow \sim 0$). At each step $i \geq 0$, the register S is updated according to the following formula

$$S \leftarrow (S \ll 1) \mid M[T[i]]. \tag{1.2}$$

The algorithm leverages the fact that every time the state register is shifted to the left, a 0 is introduced at the least significant bit. This 0 is then propagated to the left as long as the characters in the text match the characters in the pattern. If a mismatch occurs, that 0 is replaced by a 1, and the propagation stops. The state register S keeps track of the longest prefix of the pattern that matches a suffix of the text ending at position i. At step i, if $S[|P| - 1] = 0$, then there is a match ending at position i in the text. Correctness is proved by using induction to check the invariant: at step i, we have

$$S[j] = 0 \iff T[i - j, i] = P[0, j], \forall j \in [0, |P| - 1]. \tag{1.3}$$

A.2 BST-like distributions of regular expression trees

B

Bibliography

[1] Victor Mikhaylovich Glushkov. "The abstract theory of automata". In: *Russian Mathematical Surveys* 16.5 (1961), pp. 1–53.

[2] Jan Holub, William F Smyth, and Shu Wang. "Fast pattern-matching on indeterminate strings". In: *Journal of Discrete Algorithms* 6.1 (2008), pp. 37–50.

[3] Donald E Knuth. *The Art of Computer Programming: Fundamental Algorithms, Volume 1*. Addison-Wesley Professional, 1997.

[4] Florent Koechlin, Cyril Nicaud, and Pablo Rotondo. "Uniform random expressions lack expressivity". In: *44th International Symposium on Mathematical Foundations of Computer Science (MFCS 2019)*. Schloss Dagstuhl–Leibniz-Zentrum für Informatik. 2019, pp. 51–1.

[5] Kun Qiu et al. "Teddy: An efficient simd-based literal matching engine for scalable deep packet inspection". In: *Proceedings of the 50th International Conference on Parallel Processing*. 2021, pp. 1–11.

[6] Vectorcamp. *Vectorscan: A portable fork of the high-performance regular expression matching library*. https://github.com/Vectorcamp/vectorscan. 2023.

[7] Xiang Wang et al. "Hyperscan: A fast multi-pattern regex matcher for modern {CPUs}". In: *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*. 2019, pp. 631–648.

[8] Hao Xu et al. "Harry: A scalable SIMD-based multi-literal pattern matching engine for deep packet inspection". In: *IEEE INFOCOM 2023-IEEE Conference on Computer Communications*. IEEE. 2023, pp. 1–10.